



AALBORG  
UNIVERSITY

# Database Systems

## *Lecture 10: Relational Database Design*

**Yumeng Song**

Department of Computer Science

Aalborg University

[yumengs@cs.aau.dk](mailto:yumengs@cs.aau.dk)

Fall 2025

# Motivation

---



- A good design prevents problems that occur later during usage
- Changing a database schema before it is in use is much cheaper
- Redundancy is the source of many problems and should be avoided by meeting certain design criteria

# Learning Objectives

---



- After completing this lecture, you will be able to:
  - Understand concepts of functional dependencies and normal forms
  - Describe the quality of a design by using normal forms
  - Improve a database design by decomposition of relations

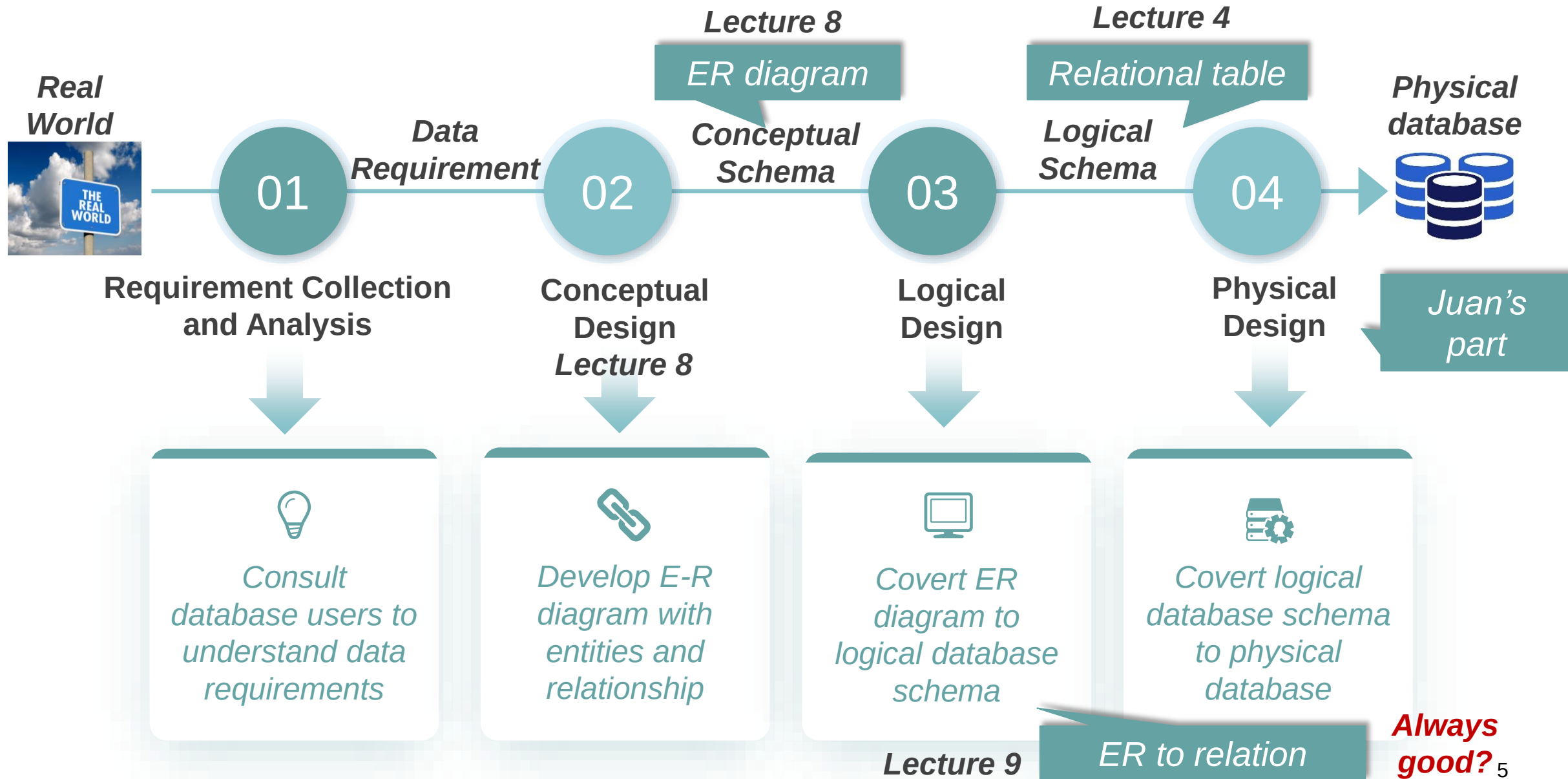
# Agenda

---



- Introduction of normalization theory
- Functional dependency
- Attribute closure
- Keys
- Decomposition
- Normal forms

# Steps of Database Design



# Problems in Relational Database Design



- Example: Instructor and Department
  - Suppose we want to manage instructors and their departments.
  - If we design a relation like this:

*in\_dep (i\_id, name, salary, dept\_name, dept\_building, dept\_budget)*

| <u>i_id</u> | name       | salary | dept_name  | dept_building | dept_budget |
|-------------|------------|--------|------------|---------------|-------------|
| 10101       | Srinivasan | 65000  | Comp. Sci. | Taylor        | 100000      |
| 12121       | Wu         | 90000  | Finance    | Painter       | 120000      |
| 45565       | Katz       | 75000  | Comp. Sci. | Taylor        | 100000      |
| 58583       | Califieri  | 62000  | History    | Painter       | 50000       |
| 76543       | Singh      | 80000  | Finance    | Painter       | 120000      |

*Any problem?*

# Problems in Relational Database Design



- Example: Instructor and Department

*in\_dep (i\_id, name, salary, dept\_name, dept\_building, dept\_budget)*

| <u>i_id</u> | name       | salary | dept_name      | dept_building  | dept_budget   |
|-------------|------------|--------|----------------|----------------|---------------|
| 10101       | Srinivasan | 65000  | Comp. Sci.     | Taylor         | 100000        |
| 12121       | Wu         | 90000  | <b>Finance</b> | <b>Painter</b> | <b>120000</b> |
| 45565       | Katz       | 75000  | Comp. Sci.     | Taylor         | 100000        |
| 58583       | Califieri  | 62000  | History        | Painter        | 50000         |
| 76543       | Singh      | 80000  | <b>Finance</b> | <b>Painter</b> | <b>120000</b> |

- Data redundancy
  - Department information (*dept\_building*, *dept\_budget*) is repeated for every instructor in the same department.
  - Example: “Finance” department appears **TWICE**, with the same building and budget.

# Problems in Relational Database Design



- Example: Instructor and Department

*in\_dep (i\_id, name, salary, dept\_name, dept\_building, dept\_budget)*

| <u>i_id</u> | name       | salary | dept_name      | dept_building | dept_budget                        |
|-------------|------------|--------|----------------|---------------|------------------------------------|
| 10101       | Srinivasan | 65000  | Comp. Sci.     | Taylor        | 100000                             |
| 12121       | Wu         | 90000  | <b>Finance</b> | Painter       | <del>120000</del><br><b>200000</b> |
| 45565       | Katz       | 75000  | Comp. Sci.     | Taylor        | 100000                             |
| 58583       | Califieri  | 62000  | History        | Painter       | 50000                              |
| 76543       | Singh      | 80000  | <b>Finance</b> | Painter       | <del>120000</del><br><b>200000</b> |

- Update anomaly
  - If the budget of a department changes, it must be updated in multiple rows.
  - Example: Finance department's budget changes from 120,000 to 200,000.
    - ◆ Must update **ALL** instructors in Finance
    - ◆ Missing one row causes inconsistency



# Problems in Relational Database Design



- Example: Instructor and Department

*in\_dep (i\_id, name, salary, dept\_name, dept\_building, dept\_budget)*

| i_id  | name       | salary | dept_name  | dept_building | dept_budget |
|-------|------------|--------|------------|---------------|-------------|
| 10101 | Srinivasan | 65000  | Comp. Sci. | Taylor        | 100000      |
| 12121 | Wu         | 90000  | Finance    | Painter       | 120000      |
| 45565 | Katz       | 75000  | Comp. Sci. | Taylor        | 100000      |
| 58583 | Califieri  | 62000  | History    | Painter       | 50000       |
| 76543 | Singh      | 80000  | Finance    | Painter       | 120000      |
|       |            |        | Physics    | Watson        | 150000      |



- Insertion anomaly
  - Cannot add a new department until at least one instructor is assigned.
  - Example: Want to add “Physics” department (Watson building, budget 150000)
    - ◆ **NOT** possible without inserting a dummy instructor.

# Problems in Relational Database Design



- Example: Instructor and Department

*in\_dep (i\_id, name, salary, dept\_name, dept\_building, dept\_budget)*

| <u>i_id</u>  | name             | salary       | dept_name      | dept_building  | dept_budget  |
|--------------|------------------|--------------|----------------|----------------|--------------|
| 10101        | Srinivasan       | 65000        | Comp. Sci.     | Taylor         | 100000       |
| 12121        | Wu               | 90000        | Finance        | Painter        | 120000       |
| 45565        | Katz             | 75000        | Comp. Sci.     | Taylor         | 100000       |
| <b>58583</b> | <b>Califieri</b> | <b>62000</b> | <b>History</b> | <b>Painter</b> | <b>50000</b> |
| 76543        | Singh            | 80000        | Finance        | Painter        | 120000       |

- Deletion anomaly
  - If the last instructor in a department is deleted, department information is lost.
  - Example: If Califieri (History) leaves,
    - ◆ All records about the Califieri department (budget, building) **DISAPPEAR**.

# Solution to Anomalies: Decomposition



- Example: Instructor and Department

*in\_dep (i\_id, name, salary, dept\_name, dept\_building, dept\_budget)*

| <u>i_id</u> | name       | salary | dept_name  | dept_building | dept_budget |
|-------------|------------|--------|------------|---------------|-------------|
| 10101       | Srinivasan | 65000  | Comp. Sci. | Taylor        | 100000      |
| 12121       | Wu         | 90000  | Finance    | Painter       | 120000      |
| 45565       | Katz       | 75000  | Comp. Sci. | Taylor        | 100000      |
| 58583       | Califieri  | 62000  | History    | Painter       | 50000       |
| 76543       | Singh      | 80000  | Finance    | Painter       | 120000      |

- A **good** case -- decompose *in\_dep* into:

*Instructor (i\_id, name, salary, dept\_name → dept)*

| <u>i_id</u> | name       | salary | dept_name  |
|-------------|------------|--------|------------|
| 10101       | Srinivasan | 65000  | Comp. Sci. |
| 12121       | Wu         | 90000  | Finance    |
| 45565       | Katz       | 75000  | Comp. Sci. |
| 58583       | Califieri  | 62000  | History    |
| 76543       | Singh      | 80000  | Finance    |

*dept (dept\_name, building, budget)*

| <u>dept_name</u> | building | budget |
|------------------|----------|--------|
| Comp. Sci.       | Taylor   | 100000 |
| Finance          | Painter  | 120000 |
| History          | Painter  | 50000  |

# Solution to Anomalies: Decomposition



- Example: Instructor and Department

*in\_dep (i\_id, name, salary, dept\_name, dept\_building, dept\_budget)*

| <u>i_id</u> | name       | salary | dept_name  | dept_building | dept_budget |
|-------------|------------|--------|------------|---------------|-------------|
| 10101       | Srinivasan | 65000  | Comp. Sci. | Taylor        | 100000      |
| 12121       | Wu         | 90000  | Finance    | Painter       | 120000      |
| 45565       | Katz       | 75000  | Comp. Sci. | Taylor        | 100000      |
| 58583       | Califieri  | 62000  | History    | Painter       | 50000       |
| 76543       | Singh      | 80000  | Finance    | Painter       | 120000      |

- A **bad** case -- decompose *in\_dep* into:

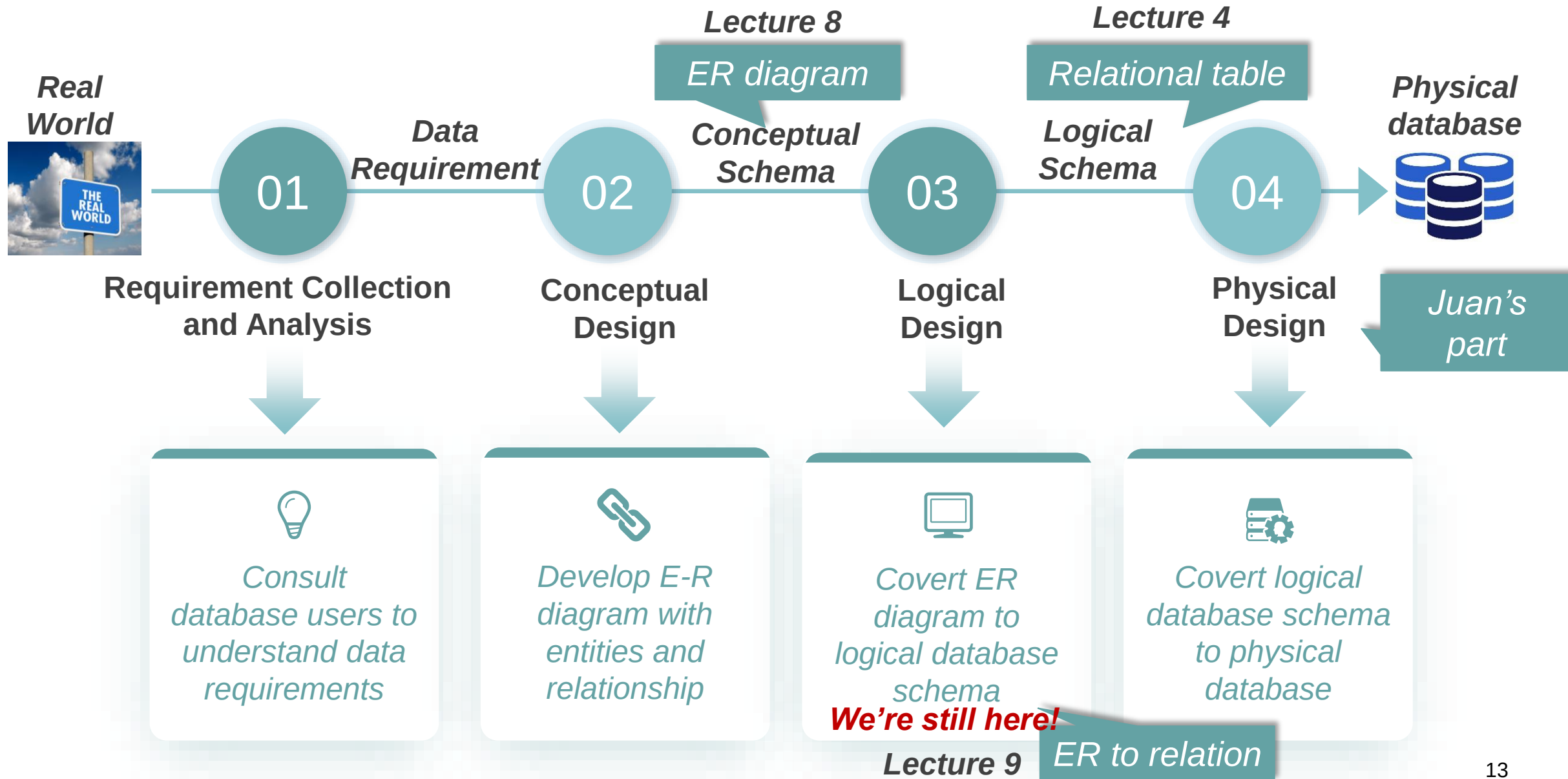
| <u>i_id</u> | name       | salary | <u>dept_name</u> | building | budget |
|-------------|------------|--------|------------------|----------|--------|
| 10101       | Srinivasan | 65000  | Comp. Sci.       | Taylor   | 100000 |
| 12121       | Wu         | 90000  | Finance          | Painter  | 120000 |
| 45565       | Katz       | 75000  | History          | Painter  | 50000  |
| 58583       | Califieri  | 62000  |                  |          |        |
| 76543       | Singh      | 80000  |                  |          |        |

*Instructor (i\_id, name, salary)*

*department(dept\_name,  
building, budget)*

- 
- Loses the link between instructor and department
  - Does not preserve dependencies
  - Cannot reconstruct original relation

# Steps of Database Design



# Normalization Theory

---



- Relational database design problems
  - *Given a relation schema, how do we decide whether it has problems?*
  - *If it does, how many relations should we decompose it into?*
  - *Which attributes belong to each relation?*

This motivates **Normalization Theory**

- Normalization
  - A process of **restructuring relation schemas** to control redundancy and anomalies.
  - Decompose a relation into smaller relations such that
    - ◆ Each relation is in **good** form
    - ◆ The decomposition is a **lossless** decomposition (no information is lost, no spurious tuples)
  - Normalization relies on **Functional Dependencies (FDs)**
    - ◆ FDs guide decomposition: detect redundancy and decide how to split relations

# Agenda

---



- Introduction of normalization theory
- Functional dependency
- Attribute sets closure
- Keys
- Decomposition
- Normal forms

# Functional Dependencies (FDs)

---



- There are usually a variety of constraints (rules) on the data in the real world
  - Real-world constraints in a university database:
    - ◆ Each student/instructor has a unique ID
    - ◆ Each ID corresponds to exactly one name
    - ◆ Each student/instructor belongs to one department
    - ◆ Each department has exactly one budget and one building
  - Some attributes determine the values of other attributes.
    - ◆  $ID \rightarrow Name$
    - ◆  $Department\ name \rightarrow Building, Budget$
- Functional Dependencies (FDs)
  - Require that the value for a certain set of attributes determines uniquely the value for another set of attributes.
  - An FD is a generalization of the notion of a key.



# Functional Dependencies (FDs) Definition



- Formal Definition

- Let  $R$  be a relation schema
- Let  $r(R)$  be a legal instance of  $R$
- Let  $\alpha \subseteq R$  and  $\beta \subseteq R$  be sets of attributes
- The functional dependency  $\alpha \rightarrow \beta$  holds on  $R$  if:  
$$\forall t_1, t_2 \in r(R): t_1.\alpha = t_2.\alpha \Rightarrow t_1.\beta = t_2.\beta$$

## Example

- A schema:  $R=(A,B,C)$   
# A table with three attributes
- An instance:  $r(R) = r(A,B,C)$
- All possible sets of attributes:  
 $\{A\}, \{B\}, \{C\}, \{A,B\}, \{A,C\},$   
 $\{B,C\}, \{A,B,C\}$

- Intuitive meaning

- If two tuples have the same values on  $\alpha$ , then they must also have the same values on  $\beta$ .
- Example:
  - ◆  $ID \rightarrow name$
  - ◆ If two rows have the same ID, they must have the same name.
- If  $\beta \subseteq \alpha$ ,  $\alpha \rightarrow \beta$  is called trivial.

# Functional Dependencies (FDs) Definition



- Formal Definition

- Let  $R$  be a relation schema
- Let  $r(R)$  be a legal instance of  $R$
- Let  $\alpha \subseteq R$  and  $\beta \subseteq R$  be sets of attributes
- The functional dependency  $\alpha \rightarrow \beta$  holds on  $R$  if:  
$$\forall t_1, t_2 \in r(R): t_1.\alpha = t_2.\alpha \Rightarrow t_1.\beta = t_2.\beta$$

- Example

- Consider  $r(A, B, C)$  with the following instance of  $r$ .

| A | B | C |
|---|---|---|
| a | 4 | 5 |
| a | 4 | 6 |
| b | 1 | 2 |
| c | 8 | 3 |
| d | 1 | 5 |

$\{A\} \rightarrow \{B\}$  holds

$\{B\} \rightarrow \{A\}$  does **NOT** hold

## Example

- A schema:  $R=(A,B,C)$   
*# A table with three attributes*
- An instance:  $r(R) = r(A,B,C)$
- All possible sets of attributes:  
 $\{A\}, \{B\}, \{C\}, \{A,B\}, \{A,C\},$   
 $\{B,C\}, \{A,B,C\}$

# Functional Dependencies (FDs) Definition



- Formal Definition

- Let  $R$  be a relation schema
- Let  $r(R)$  be a legal instance of  $R$
- Let  $\alpha \subseteq R$  and  $\beta \subseteq R$  be sets of attributes
- The functional dependency  $\alpha \rightarrow \beta$  holds on  $R$  if:  
$$\forall t_1, t_2 \in r(R): t_1.\alpha = t_2.\alpha \implies t_1.\beta = t_2.\beta$$

- Example

- Which FDs hold?
  - $\{ID\} \rightarrow \{name\}$
  - ✗  $\{name\} \rightarrow \{salary\}$
  - $\{ID, name\} \rightarrow \{salary, dept\_name\}$
  - ✗  $\{name\} \rightarrow \{ID\}$
  - $\{ID, salary\} \rightarrow \{name\}$

| <u>i_id</u> | name       | salary | dept_name  |
|-------------|------------|--------|------------|
| 10101       | Srinivasan | 65000  | Comp. Sci. |
| 12121       | Wu         | 90000  | Finance    |
| 45565       | Katz       | 75000  | Comp. Sci. |
| 58583       | Califieri  | 62000  | History    |
| 67131       | Califieri  | 70000  | Physics    |
| 76543       | Singh      | 80000  | Finance    |

# Functional Dependencies (FDs) Definition



- Formal Definition

- Let  $R$  be a relation schema
- Let  $r(R)$  be a legal instance of  $R$
- Let  $\alpha \subseteq R$  and  $\beta \subseteq R$  be sets of attributes
- The functional dependency  $\alpha \rightarrow \beta$  holds on  $R$  if:  
$$\forall t_1, t_2 \in r(R): t_1.\alpha = t_2.\alpha \implies t_1.\beta = t_2.\beta$$

- Example

- Which FDs are trivial?
  - $\{ID, name\} \rightarrow \{name\}$
  - $\{ID, name, salary\} \rightarrow \{salary\}$
  - ~~$\{ID, name\} \rightarrow \{salary, dept\_name\}$~~
  - ~~$\{ID, salary\} \rightarrow \{name, salary\}$~~
  - ~~$\{ID, name, salary\} \rightarrow \{salary, dept\_name\}$~~

| <u>i_id</u> | name       | salary | dept_name  |
|-------------|------------|--------|------------|
| 10101       | Srinivasan | 65000  | Comp. Sci. |
| 12121       | Wu         | 90000  | Finance    |
| 45565       | Katz       | 75000  | Comp. Sci. |
| 58583       | Califieri  | 62000  | History    |
| 67131       | Califieri  | 70000  | Physics    |
| 76543       | Singh      | 80000  | Finance    |

If  $\beta \subseteq \alpha$ ,  $\alpha \rightarrow \beta$  is called **trivial**.

# Armstrong's Axioms

---



- Reflexive rule
  - A **set** of attributes  $\rightarrow$  A **subset** of the attributes
  - If  $\beta \subseteq \alpha$ , then  $\alpha \rightarrow \beta$
- Example
  - $\{ID, name, salary\} \rightarrow \{name\}$
  - $\{ID, name, salary\} \rightarrow \{salary\}$
  - $\{ID, name, salary\} \rightarrow \{ID, name\}$
  - $ABCD \rightarrow ABC$
  - $ABCD \rightarrow BCD$
  - $ABCD \rightarrow ACD$
  - $ABCD \rightarrow BC$
  - $ABCD \rightarrow A$
  - $ABCD \rightarrow C$

*Note: ABCD is a shorthand notation for  $\{A, B, C, D\}$*

# Armstrong's Axioms

---



- Augmentation rule
  - If  $\alpha \rightarrow \beta$ , then  $\gamma\alpha \rightarrow \gamma\beta$
- Example
  - Given  $\{ID\} \rightarrow \{name\}$ 
    - ◆  $\{ID, salary\} \rightarrow \{name, salary\}$
    - ◆  $\{ID, dept\_name\} \rightarrow \{name, dept\_name\}$
  - Given  $A \rightarrow B$ 
    - ◆  $AC \rightarrow BC$
    - ◆  $AD \rightarrow BD$
    - ◆  $ACD \rightarrow BCD$
    - ◆  $ACD \rightarrow ABCD$

# Armstrong's Axioms

---



- Transitivity rule
  - If  $\alpha \rightarrow \beta$  and  $\beta \rightarrow \gamma$ , then  $\alpha \rightarrow \gamma$
- Example
  - Given  $\{ID\} \rightarrow \{dept\_name\}$  and  $\{dept\_name\} \rightarrow \{building\}$ ,
    - ◆  $\{ID\} \rightarrow \{building\}$
  - Given  $A \rightarrow B$ ,  $B \rightarrow C$ , and  $C \rightarrow D$ 
    - ◆  $A \rightarrow C$
    - ◆  $A \rightarrow D$
    - ◆  $B \rightarrow D$

# Reasoning with FDs

---



- Example
- Given  $A \rightarrow B$  and  $BC \rightarrow D$
- Prove that  $AC \rightarrow D$
- Proof:

- Step 1

Augmentation rule: If  $\alpha \rightarrow \beta$ , then  $\gamma\alpha \rightarrow \gamma\beta$

Given  $A \rightarrow B$ , we have  $AC \rightarrow BC$

- Step 2

Transitivity rule: If  $\alpha \rightarrow \beta$  and  $\beta \rightarrow \gamma$ , then  $\alpha \rightarrow \gamma$

Given  $AC \rightarrow BC$  and  $BC \rightarrow D$ , we have  $AC \rightarrow D$



# Exercise 1

---



- Q1. Given  $A \rightarrow B$  and  $D \rightarrow C$ , prove that  $AD \rightarrow BC$
- Q2. Given  $A \rightarrow C$ ,  $AC \rightarrow D$ ,  $AD \rightarrow B$ , prove that  $A \rightarrow B$

Reflexive rule: If  $\beta \subseteq \alpha$ , then  $\alpha \rightarrow \beta$

Augmentation rule: If  $\alpha \rightarrow \beta$ , then  $\gamma\alpha \rightarrow \gamma\beta$

Transitivity rule: If  $\alpha \rightarrow \beta$  and  $\beta \rightarrow \gamma$ , then  $\alpha \rightarrow \gamma$

# Exercise 1 -- Solution

---



- *Q1. Given  $A \rightarrow B$  and  $D \rightarrow C$ , prove that  $AD \rightarrow BC$*
- *Q2. Given  $A \rightarrow C$ ,  $AC \rightarrow D$ ,  $AD \rightarrow B$ , prove that  $A \rightarrow B$*

Reflexive rule: If  $\beta \subseteq \alpha$ , then  $\alpha \rightarrow \beta$

Augmentation rule: If  $\alpha \rightarrow \beta$ , then  $\gamma\alpha \rightarrow \gamma\beta$

Transitivity rule: If  $\alpha \rightarrow \beta$  and  $\beta \rightarrow \gamma$ , then  $\alpha \rightarrow \gamma$

- *A1. Proof*
  - *Given  $A \rightarrow B$ , we have  $AD \rightarrow BD$  (Augmentation)*
  - *Given  $D \rightarrow C$ , we have  $BD \rightarrow BC$  (Augmentation)*
  - *Given  $AD \rightarrow BD$  and  $BD \rightarrow BC$ , we have  $AD \rightarrow BC$  (Transitivity)*

# Exercise 1 -- Solution



- Q1. Given  $A \rightarrow B$  and  $D \rightarrow C$ , prove that  $AD \rightarrow BC$
- Q2. Given  $A \rightarrow C$ ,  $AC \rightarrow D$ ,  $AD \rightarrow B$ , prove that  $A \rightarrow B$

Reflexive rule: If  $\beta \subseteq \alpha$ , then  $\alpha \rightarrow \beta$

Augmentation rule: If  $\alpha \rightarrow \beta$ , then  $\gamma\alpha \rightarrow \gamma\beta$

Transitivity rule: If  $\alpha \rightarrow \beta$  and  $\beta \rightarrow \gamma$ , then  $\alpha \rightarrow \gamma$

- A2. Proof
  - Given  $A \rightarrow C$ , we have  $A \rightarrow AC$  (Augmentation)
  - Given  $A \rightarrow AC$  and  $AC \rightarrow D$ , we have  $A \rightarrow D$  (Transitivity)
  - Given  $A \rightarrow D$ , we have  $A \rightarrow AD$  (Augmentation)
  - Given  $A \rightarrow AD$  and  $AD \rightarrow B$ , we have  $A \rightarrow B$  (Transitivity)

# Additional Rules



- Union rule
  - If  $\alpha \rightarrow \beta$  and  $\alpha \rightarrow \gamma$ , then  $\alpha \rightarrow \beta\gamma$
- Example
  - Given  $\{ID\} \rightarrow \{name\}$  and  $\{ID\} \rightarrow \{salary\}$ ,
    - ◆  $\{ID\} \rightarrow \{name, salary\}$

Reflexive rule: If  $\beta \subseteq \alpha$ , then  $\alpha \rightarrow \beta$

Augmentation rule: If  $\alpha \rightarrow \beta$ , then  $\gamma\alpha \rightarrow \gamma\beta$

Transitivity rule: If  $\alpha \rightarrow \beta$  and  $\beta \rightarrow \gamma$ , then  $\alpha \rightarrow \gamma$

- Proof
  - Given  $\alpha \rightarrow \beta$ , we have  $\alpha \rightarrow \alpha\beta$  (Augmentation)
  - Given  $\alpha \rightarrow \gamma$ , we have  $\alpha\beta \rightarrow \beta\gamma$  (Augmentation)
  - Given  $\alpha \rightarrow \alpha\beta$  and  $\alpha\beta \rightarrow \beta\gamma$ , we have  $\alpha \rightarrow \beta\gamma$  (Transitivity)

# Additional Rules



- Decomposition rule
  - If  $\alpha \rightarrow \beta\gamma$ , then  $\alpha \rightarrow \beta$  holds and  $\alpha \rightarrow \gamma$  holds
- Example
  - Given  $\{ID\} \rightarrow \{name, salary\}$ 
    - ◆  $\{ID\} \rightarrow \{name\}$
    - ◆  $\{ID\} \rightarrow \{salary\}$
- Proof
  - Given  $\beta\gamma$ , we have  $\beta\gamma \rightarrow \beta$  and  $\beta\gamma \rightarrow \gamma$  (Reflexive)
  - Given  $\alpha \rightarrow \beta\gamma$  and  $\beta\gamma \rightarrow \beta$ , we have  $\alpha \rightarrow \beta$  (Transitivity)
  - Given  $\alpha \rightarrow \beta\gamma$  and  $\beta\gamma \rightarrow \gamma$ , we have  $\alpha \rightarrow \gamma$  (Transitivity)

Reflexive rule: If  $\beta \subseteq \alpha$ , then  $\alpha \rightarrow \beta$

Augmentation rule: If  $\alpha \rightarrow \beta$ , then  $\gamma\alpha \rightarrow \gamma\beta$

Transitivity rule: If  $\alpha \rightarrow \beta$  and  $\beta \rightarrow \gamma$ , then  $\alpha \rightarrow \gamma$

# Additional Rules

---



- Pseudotransitivity rule
  - If  $\alpha \rightarrow \beta$  holds and  $\gamma\beta \rightarrow \delta$  holds, then  $\alpha\gamma \rightarrow \delta$
- Proof
  - Given  $\alpha \rightarrow \beta$ , we have  $\alpha\gamma \rightarrow \gamma\beta$  (Augmentation)
  - Given  $\alpha\gamma \rightarrow \gamma\beta$  and  $\gamma\beta \rightarrow \delta$ , we have  $\alpha\gamma \rightarrow \delta$  (Transitivity)

Reflexive rule: If  $\beta \subseteq \alpha$ , then  $\alpha \rightarrow \beta$

Augmentation rule: If  $\alpha \rightarrow \beta$ , then  $\gamma\alpha \rightarrow \gamma\beta$

Transitivity rule: If  $\alpha \rightarrow \beta$  and  $\beta \rightarrow \gamma$ , then  $\alpha \rightarrow \gamma$

# Agenda

---



- Introduction of normalization theory
- Functional dependency
- Attribute closure
- Keys
- Decomposition
- Normal forms

# Why Attribute Closure?

---



- Normalization relies on FDs
- Next question: What can we do with FDs?
- Attribute closure is the tool.
  - Check if an FD holds in a relation schema
  - Find candidate keys (essential for normalization)
- FDs give us the rules.
- Attribute closure lets us **apply** the rules to analyze and improve schema design.



# Closure of Attribute Sets

---



- Given:
  - A set of attributes  $\alpha$  and a FD set  $F$
- The closure of  $\alpha$  under  $F$  (denoted by  $\alpha^+$ ) is:
  - The set of attributes that are functionally determined by  $\alpha$
  - According to the dependencies in  $F$ .
- Example
  - Given:
    - ◆  $R=(A,B,C,G,H,I)$
    - ◆  $F=\{A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H\}$
  - The closure of  $AG$  under  $F$  is:
    - ◆  $(AG)^+ = ABCGHI$

# Algorithm to Compute Closure of Attribute Sets



- Pseudocode:

---

## Algorithm: attrClosure

---

Input: FD set  $F$ , attribute set  $\alpha$

Output: Closure  $\alpha^+$

1.  $\alpha^+ \leftarrow \alpha$

2. **while** ( $\alpha^+$  changes) **do**

3.     **for each**  $\beta \rightarrow \gamma$  **in**  $F$  **do**

4.         **if**  $\beta \subseteq \alpha^+$  **then**

5.              $\alpha^+ = \alpha^+ \cup \gamma$

6.         **end if**

7.     **end for**

8. **return**  $\alpha^+$

---

*Start with  $\alpha$  itself, since  $\alpha$  determines itself.*

*If the left side of FD is already in result, then we can add the right side attributes into result*

# Algorithm to Compute Closure of Attribute Sets



- Running example -- Compute  $(AG)^+$ 
  - $R=(A,B,C,G,H,I)$
  - $F=\{A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H\}$

---

## Algorithm: attrClosure

Input: FD set  $F$ , attribute set  $\alpha$

Output: Closure  $\alpha^+$

➔ 1.  $\alpha^+ \leftarrow \alpha$

2. **while** ( $\alpha^+$  changes) **do**

3.     **for each**  $\beta \rightarrow \gamma$  **in**  $F$  **do**

4.         **if**  $\beta \subseteq \alpha^+$  **then**

5.              $\alpha^+ = \alpha^+ \cup \gamma$

6.         **end if**

7.     **end for**

8. **return**  $\alpha^+$

---

$(AG)^+ = \{\underline{AG} \underline{BCHI}\}$

$A \rightarrow B$   
 $A \rightarrow C$   
 $CG \rightarrow H$   
 $CG \rightarrow I$   
 $B \rightarrow H$

# Algorithm to Compute Closure of Attribute Sets



- Running example -- Compute  $(BC)^+$ 
  - $R=(A,B,C,D,E,F)$
  - $F=\{AB \rightarrow C, AD \rightarrow E, B \rightarrow D, AF \rightarrow B\}$

## Algorithm: attrClosure

Input: FD set  $F$ , attribute set  $\alpha$

Output: Closure  $\alpha^+$

- $\alpha^+ \leftarrow \alpha$
- while** ( $\alpha^+$  changes) **do**
- for each**  $\beta \rightarrow \gamma$  **in**  $F$  **do**
- if**  $\beta \subseteq \alpha^+$  **then**
- $\alpha^+ = \alpha^+ \cup \gamma$
- end if**
- end for**
- return**  $\alpha^+$

## Process

- Initialize:  $(BC)^+ = \{BC\}$
  - Iteration 1:
    - $(BC)^+ = \{BC\}$  ( $AB \rightarrow C$ )
    - $(BC)^+ = \{BC\}$  ( $AD \rightarrow E$ )
    - $(BC)^+ = \{BCD\}$  ( $B \rightarrow D$ )
    - $(BC)^+ = \{BCD\}$  ( $AF \rightarrow B$ )
  - Iteration 2:
    - $(BC)^+ = \{BCD\}$  ( $AB \rightarrow C$ )
    - $(BC)^+ = \{BCD\}$  ( $AD \rightarrow E$ )
    - $(BC)^+ = \{BCD\}$  ( $B \rightarrow D$ )
    - $(BC)^+ = \{BCD\}$  ( $AF \rightarrow B$ )
- Has changes, continues running
- No changes

- Result  $(BC)^+ = \{BCD\}$

# Exercise 2



- Given  $R=(A,B,C,D,E,F)$  and  $F=\{AB \rightarrow C, AD \rightarrow E, B \rightarrow D, AF \rightarrow B\}$
- Compute the closure of the attribute set:  $(AF)^+$

---

## Algorithm: attrClosure

---

Input: FD set  $F$ , attribute set  $\alpha$

Output: Closure  $\alpha^+$

1.  $\alpha^+ \leftarrow \alpha$
  2. **while** ( $\alpha^+$  changes) **do**
  3.     **for each**  $\beta \rightarrow \gamma$  **in**  $F$  **do**
  4.         **if**  $\beta \subseteq \alpha^+$  **then**
  5.              $\alpha^+ = \alpha^+ \cup \gamma$
  6.         **end if**
  7.     **end for**
  8. **return**  $\alpha^+$
-

# Exercise 2 -- Solution



- Given  $R=(A,B,C,D,E,F)$  and  $F=\{AB \rightarrow C, AD \rightarrow E, B \rightarrow D, AF \rightarrow B\}$
- Compute the closure of the attribute set:  $(AF)^+$

## Algorithm: attrClosure

Input: FD set  $F$ , attribute set  $\alpha$

Output: Closure  $\alpha^+$

1.  $\alpha^+ \leftarrow \alpha$
2. **while** ( $\alpha^+$  changes) **do**
3.     **for each**  $\beta \rightarrow \gamma$  **in**  $F$  **do**
4.         **if**  $\beta \subseteq \alpha^+$  **then**
5.              $\alpha^+ = \alpha^+ \cup \gamma$
6.         **end if**
7.     **end for**
8. **return**  $\alpha^+$

## Process

- Initialize:  $(AF)^+ = \{AF\}$
- Iteration 1:
  - $(AF)^+ = \{AF\}$  ( $AB \rightarrow C$ )
  - $(AF)^+ = \{AF\}$  ( $AD \rightarrow E$ )
  - $(AF)^+ = \{AF\}$  ( $B \rightarrow D$ )
  - $(AF)^+ = \{ABF\}$  ( $AF \rightarrow B$ )

Has changes,  
continues running

- Iteration 2:
  - $(AF)^+ = \{ABCF\}$  ( $AB \rightarrow C$ )
  - $(AF)^+ = \{ABCF\}$  ( $AD \rightarrow E$ )
  - $(AF)^+ = \{ABCDF\}$  ( $B \rightarrow D$ )
  - $(AF)^+ = \{ABCDF\}$  ( $AF \rightarrow B$ )

Has changes,  
continues running

# Exercise 2 -- Solution



- Given  $R=(A,B,C,D,E,F)$  and  $F=\{AB \rightarrow C, AD \rightarrow E, B \rightarrow D, AF \rightarrow B\}$
- Compute the closure of the attribute set:  $(AF)^+$

## Algorithm: attrClosure

Input: FD set  $F$ , attribute set  $\alpha$

Output: Closure  $\alpha^+$

1.  $\alpha^+ \leftarrow \alpha$
2. **while** ( $\alpha^+$  changes) **do**
3.   **for each**  $\beta \rightarrow \gamma$  **in**  $F$  **do**
4.     **if**  $\beta \subseteq \alpha^+$  **then**
5.        $\alpha^+ = \alpha^+ \cup \gamma$
6.     **end if**
7.   **end for**
8. **return**  $\alpha^+$

## Process

- Iteration 2 result:  $(AF)^+ = \{ABCDF\}$
- Iteration 3:
  - $(AF)^+ = \{ABCDF\}$  ( $AB \rightarrow C$ )
  - $(AF)^+ = \{ABCDEF\}$  ( $AD \rightarrow E$ ) *Has changes*
  - $(AF)^+ = \{ABCDEF\}$  ( $B \rightarrow D$ )
  - $(AF)^+ = \{ABCDEF\}$  ( $AF \rightarrow B$ )
- Iteration 4:
  - $(AF)^+ = \{ABCDEF\}$  ( $AB \rightarrow C$ )
  - $(AF)^+ = \{ABCDEF\}$  ( $AD \rightarrow E$ )
  - $(AF)^+ = \{ABCDEF\}$  ( $B \rightarrow D$ )
  - $(AF)^+ = \{ABCDEF\}$  ( $AF \rightarrow B$ ) *No changes, stop running*

- Result  $(AF)^+ = \{ABCDEF\}$

# Closure & FD

---



- To prove that  $X \rightarrow Y$  holds, we only need to show that  $(X)^+$  contains  $Y$ .
- Example:
  - $R = (A, B, C, D, E, F)$
  - $F = \{AB \rightarrow C, AD \rightarrow E, B \rightarrow D, AF \rightarrow B\}$
  - Prove that  $AF \rightarrow D$
- $(AF)^+ = \{ABCDEF\}$  (see exercise 2), which contains  $D$
- Therefore,  $AF \rightarrow D$  holds



# Agenda

---



- Introduction of normalization theory
- Functional dependency (FD)
- Attribute closure
- Keys
- Decomposition
- Normal forms

# Identifying Keys Using FDs

---



- Superkey
  - A set of attributes in a table that decides all other attributes
  - $K$  is a superkey for relation schema  $R$  if and only if  $K \rightarrow R$
- Candidate key
  - A minimal superkey
  - $K$  is a candidate key for  $R$  if and only if
    - ◆  $K \rightarrow R$ : A candidate key must be a superkey
    - ◆ For no  $\alpha \subset K$  ( $\alpha \rightarrow R$ ): If we remove any attribute from the candidate key, it will not decide all other attributes
- Primary key
  - Choose one from the candidate keys

# Identifying Keys Using FDs



- Superkey
  - $K$  is a superkey for relation schema  $R$  if and only if  $K \rightarrow R$
- Candidate key
  - $K$  is a candidate key for  $R$  if and only if
    - ◆  $K \rightarrow R$ : A candidate key must be a superkey
    - ◆ For no  $\alpha \subset K$  ( $\alpha \rightarrow R$ ): If we remove any attribute from the candidate key, it will not decide all other attributes
- Example

$R = (A, B, C, G, H, I)$

$F = \{A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H\}$



$(AG)^+ = \{ABCGHI\}$

- Is  $AG$  a superkey? **Yes!**
- Is  $AG$  a candidate key? **Yes!**
  - ◆  $AG$  is a superkey
  - ◆ No subset of  $AG$  is a superkey:  $(A)^+ = BCH$  and  $(G)^+ = G$

# Finding the Candidate Keys



- How can we efficiently find all candidate keys in a schema?
- Use FDs and closures

## Algorithm

1. Check all possible combinations of attributes in the table
2. For each combination, compute its closure
3. If a closure contains all attributes, then the combination is a superkey, and might be a candidate key
4. Make sure that you select candidate keys

Example:  $R=(A,B,C)$   $F=\{A \rightarrow B\}$

1.  $A, B, C, AB, AC, BC, ABC$
2.  $(A)^+ = \{AB\}$ ,  $(B)^+ = \{B\}$ ,  $(C)^+ = \{C\}$ ,  $(AB)^+ = \{AB\}$ ,  
 $(AC)^+ = \{ABC\}$ ,  $(BC)^+ = \{BC\}$ ,  $(ABC)^+ = \{ABC\}$
3. Superkeys:  $AC, ABC$
4. Candidate keys:  
 $AC$  is a candidate key  $((A)^+ = \{AB\}, (C)^+ = \{C\})$   
 $ABC$  is NOT a candidate key  $((AC)^+ = \{ABC\})$

# Finding the Candidate Keys



- How can we efficiently find all candidate keys in a schema?
- Use FDs and closures

## Algorithm

1. Check all possible combinations of attributes in the table
2. For each combination, compute its closure
3. If a closure contains all attributes, then the combination is a superkey, and might be a candidate key
4. Make sure that you select candidate keys

## Example

$R=(A,B,C,D)$

$F=\{AB \rightarrow C, AD \rightarrow B, B \rightarrow D\}$

1. *Enumerate all attribute combinations*
  - $A, B, C, D$
  - $AB, AC, AD, BC, BD, CD$
  - $ABC, ABD, ACD, BCD$
  - $ABCD$

# Finding the Candidate Keys



- How can we efficiently find all candidate keys in a schema?
- Use FDs and closures

## Algorithm

1. Check all possible combinations of attributes in the table
2. For each combination, compute its closure
3. If a closure contains all attributes, then the combination is a superkey, and might be a candidate key
4. Make sure that you select candidate keys

## Example

$R=(A,B,C,D)$

$F=\{AB \rightarrow C, AD \rightarrow B, B \rightarrow D\}$

### 2. Compute the closures:

- $(A)^+ = \{A\}, (B)^+ = \{BD\}, (C)^+ = \{C\}, (D)^+ = \{D\}$
- $(AB)^+ = \{ABCD\}, (AC)^+ = \{AC\}, (AD)^+ = \{ABCD\}, (BC)^+ = \{BCD\}, (BD)^+ = \{BD\}, (CD)^+ = \{CD\}$
- $(ABC)^+ = \{ABCD\}, (ABD)^+ = \{ABCD\}, (ACD)^+ = \{ABCD\}, (BCD)^+ = \{BCD\}$
- $(ABCD)^+ = \{ABCD\}$

# Finding the Candidate Keys



- How can we efficiently find all candidate keys in a schema?
- Use FDs and closures

## Algorithm

1. Check all possible combinations of attributes in the table
2. For each combination, compute its closure
3. If a closure contains all attributes, then the combination is a superkey, and might be a candidate key
4. Make sure that you select candidate keys

## Example

$R=(A,B,C,D)$

$F=\{AB \rightarrow C, AD \rightarrow B, B \rightarrow D\}$

3. Find all superkeys:  **$AB, AD, ABC, ABD, ACD, ABCD$** 
  - $(A)^+ = \{A\}, (B)^+ = \{BD\}, (C)^+ = \{C\}, (D)^+ = \{D\}$
  - **$(AB)^+ = \{ABCD\}$** ,  $(AC)^+ = \{AC\}$ ,  **$(AD)^+ = \{ABCD\}$** ,  $(BC)^+ = \{BCD\}$ ,  $(BD)^+ = \{BD\}$ ,  $(CD)^+ = \{CD\}$
  - **$(ABC)^+ = \{ABCD\}$** ,  **$(ABD)^+ = \{ABCD\}$** ,  **$(ACD)^+ = \{ABCD\}$** ,  $(BCD)^+ = \{BCD\}$
  - **$(ABCD)^+ = \{ABCD\}$**

# Finding the Candidate Keys



- How can we efficiently find all candidate keys in a schema?
- Use FDs and closures

## Algorithm

1. Check all possible combinations of attributes in the table
2. For each combination, compute its closure
3. If a closure contains all attributes, then the combination is a superkey, and might be a candidate key
4. Make sure that you select candidate keys

## Example

$R=(A,B,C,D)$

$F=\{AB \rightarrow C, AD \rightarrow B, B \rightarrow D\}$

4. Select candidate keys: **AB** and **AD**
  - $(A)^+ = \{A\}, (B)^+ = \{BD\}, (C)^+ = \{C\}, (D)^+ = \{D\}$
  - **$(AB)^+ = \{ABCD\}$** ,  $(AC)^+ = \{AC\}$ ,  **$(AD)^+ = \{ABCD\}$** ,  $(BC)^+ = \{BCD\}, (BD)^+ = \{BD\}, (CD)^+ = \{CD\}$
  - **$(ABC)^+ = \{ABCD\}$** ,  **$(ABD)^+ = \{ABCD\}$** ,  **$(ACD)^+ = \{ABCD\}$** ,  $(BCD)^+ = \{BCD\}$
  - **$(ABCD)^+ = \{ABCD\}$**



# Finding the Candidate Keys



- Small Trick
  - Always check small combinations first

## Algorithm

1. Check all possible combinations of attributes in the table
2. For each combination, compute its closure
3. If a closure contains all attributes, then the combination is a superkey, and might be a candidate key
4. Make sure that you select candidate keys

## Example

$R=(A,B,C,D)$

$F=\{A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow A\}$

### 2. Compute the closures

- $(A)^+ = \{ABCD\}, (B)^+ = \{ABCD\}, (C)^+ = \{ABCD\}, (D)^+ = \{ABCD\}$ 
  - No need to check others
  - The others are all superkeys but not candidate keys

- Candidate keys:  $A, B, C, D$

# Finding the Candidate Keys



- Small Trick
  - If an attribute that does not appear in the right side of any FD, then it must be in every key

## Algorithm

1. Check all possible combinations of attributes in the table
2. For each combination, compute its closure
3. If a closure contains all attributes, then the combination is a superkey, and might be a candidate key
4. Make sure that you select candidate keys

## Example

$R=(A,B,C,D)$

$F=\{AB \rightarrow C, AD \rightarrow B, B \rightarrow D\}$

- Notice that **A** does not appear in the right side of any functional dependencies
- In that case, **A** must be in every key
- Only check closures with **A**
- Candidate keys:  $AB, AD$  (See the previous slide)

# Agenda

---



- Introduction of normalization theory
- Functional dependency
- Attribute closure
- Keys
- Decomposition
- Normal forms

# Decomposition

---



- In normalization, we decompose a relational schema  $R$  into multiple relational schemas  $R_1, R_2, \dots, R_n$  to eliminate problems in the original design.
- We care about two goals:
  - Lossless decomposition
    - ◆ Data safe -- No information is lost after decomposition.
    - ◆ Example: Split student info into “Student–Dorm” and “Dorm–Address” tables
    - ◆ When joined back, we get exactly the original student info (nothing lost, nothing extra)
  - Dependency preserving
    - ◆ Rule safe -- All functional dependencies of  $R$  can still be enforced
    - ◆ Example: Split course info into “Course–Instructor” and “Instructor–Room” tables
    - ◆ We can still check the rules: each course has one instructor, each instructor has one room
- A **good** decomposition is lossless and dependency preserving.

# Lossless Decomposition



- A decomposition is a **lossless decomposition** if there is no loss of information by replacing  $R$  with the two relation schemas  $R_1 \cup R_2$
- Formally, a decomposition is **lossless** if

$$\pi_{R_1}(r) \bowtie \pi_{R_2}(r) = r$$

$r$  is an instance of  $R$

- Example: Decomposition of  $R=(A,B,C)$

- $R_1 = (A,B)$   $R_2 = (B,C)$

| A        | B | C |
|----------|---|---|
| $\alpha$ | 1 | A |
| $\beta$  | 2 | B |

$r$

| A        | B |
|----------|---|
| $\alpha$ | 1 |
| $\beta$  | 2 |

$\Pi_{A,B}(r)$

| B | C |
|---|---|
| 1 | A |
| 2 | B |

$\Pi_{B,C}(r)$

$\Pi_A(r) \bowtie \Pi_B(r)$

| A        | B | C |
|----------|---|---|
| $\alpha$ | 1 | A |
| $\beta$  | 2 | B |

# Lossless Decomposition

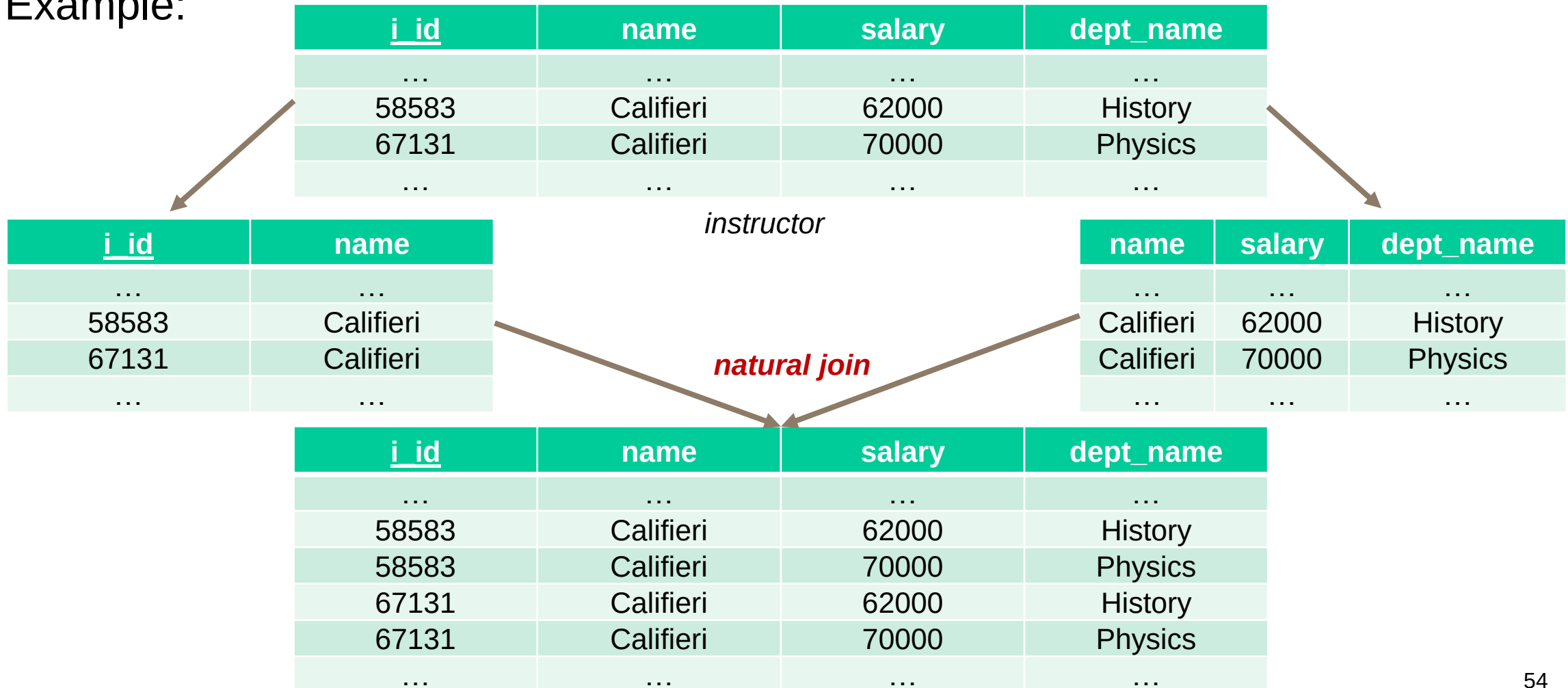


- Conversely a decomposition  $\{R_1, R_2\}$  ( $R = R_1 \cup R_2$ ) is **lossy** if

$$r \subset \pi_{R_1}(r) \bowtie \pi_{R_2}(r)$$

$r$  is an instance of  $R$

- Example:



# Checking Lossless Decomposition



- Goal: Verify whether a decomposition of a relation is lossless.
- Method: Use FDs
- Given a relation schema  $R$  and its decomposition  $R = R_1 \cup R_2$
- The decomposition  $\{R_1, R_2\}$  is lossless if **at least one** of the following holds:
  - $(R_1 \cap R_2) \rightarrow R_1$ , i.e., common attributes are superkey in  $R_1$
  - $(R_1 \cap R_2) \rightarrow R_2$ , i.e., common attributes are superkey in  $R_2$
- Example:
  - Given  $R = (A, B, C)$  and  $F = (A \rightarrow B, B \rightarrow C)$
  - Decomposition  $\{R_1 = (A, B), R_2 = (B, C)\}$ 
    - ◆ Lossless decomposition
    - ◆  $R_1 \cap R_2 = \{B\}$  and  $B \rightarrow BC$
  - Decomposition  $\{R_1 = (A, B), R_2 = (A, C)\}$ 
    - ◆ Lossless decomposition
    - ◆  $R_1 \cap R_2 = \{A\}$  and  $A \rightarrow AB$

# Exercise 3

---



- *Question: Is this decomposition lossless?*
  - department (*name, street, number, city, postcode*)
  - FDs
    - ◆  $\{name\} \rightarrow \{street, number, city, postcode\}$
    - ◆  $\{street, number, city\} \rightarrow \{postcode\}$
    - ◆  $\{street, number, city\} \rightarrow \{name\}$
  - Decomposition
    - ◆ dept1 (*name, street, number, city*)
    - ◆ dept2 (*name, postcode*)



# Exercise 3 -- Solution

---



- *Question: Is this decomposition lossless?*
  - department (*name, street, number, city, postcode*)
  - FDs
    - ◆  $\{name\} \rightarrow \{street, number, city, postcode\}$
    - ◆  $\{street, number, city\} \rightarrow \{postcode\}$
    - ◆  $\{street, number, city\} \rightarrow \{name\}$
  - Decomposition
    - ◆ dept1 (*name, street, number, city*)
    - ◆ dept2 (*name, postcode*)
- *Answer: Yes.*
  - $dept1 \cap dept2 = \{name, street, number, city\} \cap \{name, postcode\} = \{name\}$
  - $\{name\} \rightarrow \{name, street, number, city\}$  # dept1
  - $\{name\} \rightarrow \{name, postcode\}$  # dept2

# Dependency Preserving

---



- When we split a big table into smaller ones, we also need to keep FDs about the data.
- A decomposition is dependency preserving if:
  - **All** rules can still be checked in the smaller tables directly
  - We **don't need to join the tables** back together to verify them
- Example: Is this decomposition dependency preserving?
  - $R = (A, B, C)$
  - $F = \{A \rightarrow B, B \rightarrow C\}$
  - Decomposition:  $\{R_1 = (A, B), R_2 = (B, C)\}$
  - **Yes!**
  - $A \rightarrow B$  can be checked locally in  $R_1$
  - $B \rightarrow C$  can be checked locally in  $R_2$

# Agenda

---

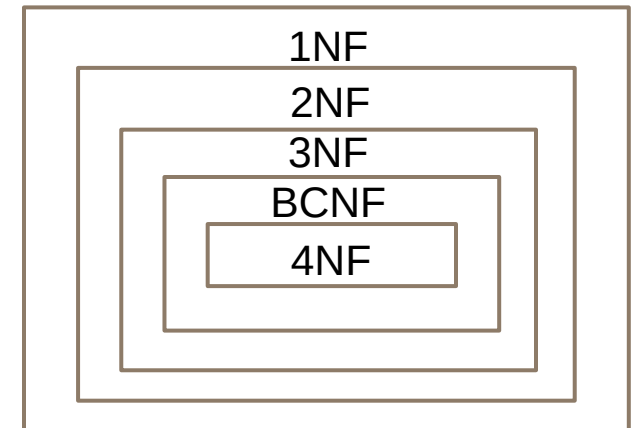


- Introduction of normalization theory
- Functional dependency
- Attribute closure
- Keys
- Decomposition
- Normal forms

# Normal Forms



- Normal forms provide the standards to evaluate whether a relational schema design is **good**.
- Normal forms provide principles to:
  - Define characteristics of relational schemas
  - Forbid certain “bad” functional dependencies
  - Help avoid redundancy and anomalies
  - Act as guidelines for good decompositions
- Levels of normal forms: **1NF, 2NF, 3NF, BCNF, 4NF**



# First Normal Forms (1NF)



- $R$  is in 1NF if the domains of all its attributes are atomic (no composite or set-valued domains)

| i_id  | name       | course_id              |
|-------|------------|------------------------|
| 10101 | Srinivasan | CS-101, CS-120, CS-131 |
| 12121 | Wu         | FIN-101                |
| 15151 | Mozart     | MUS-101, MUS-145       |
| 83821 | Brandt     | CS-190                 |
| 98345 | Kim        | EE-190                 |

Not in 1NF

| i_id  | name       | course_id |
|-------|------------|-----------|
| 10101 | Srinivasan | CS-101    |
| 10101 | Srinivasan | CS-120    |
| 10101 | Srinivasan | CS-131    |
| 12121 | Wu         | FIN-101   |
| 15151 | Mozart     | MUS-101   |
| 15151 | Mozart     | MUS-145   |
| 83821 | Brandt     | CS-190    |
| 98345 | Kim        | EE-190    |

In 1NF

# Second Normal Forms (2NF)



- $R$  is in 2NF if every non-key attribute is functionally dependent on the whole of every candidate key.
- 2NF prevents partial dependencies

| <u>i_id</u> | name       | <u>course_id</u> |
|-------------|------------|------------------|
| 10101       | Srinivasan | CS-101           |
| 10101       | Srinivasan | CS-120           |
| 10101       | Srinivasan | CS-131           |
| 12121       | Wu         | FIN-101          |
| 15151       | Mozart     | MUS-101          |
| 15151       | Mozart     | MUS-145          |
| 83821       | Brandt     | CS-190           |
| 98345       | Kim        | EE-190           |

- $i\_id \rightarrow name$  (Holds)
- $course\_id \rightarrow name$  (Does Not hold)

$name$  depends only on  
part of the candidate key

Not in 2NF

# Third Normal Forms (3NF)



- A relation schema  $R$  is in 3NF if at least one of the following conditions holds for each of its FDs  $\alpha \rightarrow \beta$  with  $\alpha, \beta \subseteq R$ :
  - $\alpha \rightarrow \beta$  is trivial (i.e.,  $\beta \subseteq \alpha$ )
  - $\alpha$  is a superkey of  $R$  ←
  - Each attribute  $A$  in  $\beta - \alpha$  is contained in a candidate key for  $R$ .  
(Note: Each attribute may be in a different candidate key)
- Example: Is this relation in 3NF?
  - Relation: *Instructor* ( $ID, name, dept\_name, building$ )
  - Candidate key:  $\{ID\}$
  - $F = \{\{ID\} \rightarrow \{name, dept\_name, building\}, \{dept\_name\} \rightarrow \{building\}\}$
  - $\{ID\} \rightarrow \{name, dept\_name, building\}$  ○
  - $\{dept\_name\} \rightarrow \{building\}$  ✗
  - Not in 3NF

# Boyce Codd Normal Form (BCNF)



- A relation schema  $R$  is in BCNF if at least one of the following conditions holds for each of its FDs  $\alpha \rightarrow \beta$  with  $\alpha, \beta \subseteq R$ :
  - $\alpha \rightarrow \beta$  is trivial (i.e.,  $\beta \subseteq \alpha$ )
  - $\alpha$  is a superkey of  $R$
  - ~~Each attribute  $A$  in  $\beta - \alpha$  is contained in a candidate key for  $R$ .~~
- Difference to 3NF: BCNF only allows dependencies with a superkey.
  - Relation:  $R(A, B, C)$
  - $F = \{AB \rightarrow C, C \rightarrow B\}$
  - Candidate key:  $\{AB, AC\}$
  - $AB \rightarrow C$  ( $AB$  is a superkey)
  - $C \rightarrow B$  ( $B$  is contained in candidate key  $AB$ )
  - $R$  is in 3NF but not in BCNF
- BCNF is a stricter form of 3NF (“includes” 3NF)



# Summary

---



- Introduction of normalization theory
- Functional dependency
- Attribute closure
- Keys
- Decomposition
- Normal forms



**Any Questions?**

***Lecture 10:***  
***Relational Database Design***

**Yumeng Song**

yumengs@cs.aau.dk

Center for Data-Intensive Systems

# Reference

---



- Abraham et al.. Database System Concepts (seventh edition) – chapter 7
- Tiantian Liu. Database Systems(SW5) - 7. Relational Database Design
- Gabriela Montoya. Database Systems - Design Theory